📄 Product Requirements Document (PRD) v1.0

Project: Smart Salon Addis

Status: Draft / Initial Review

Target Date: 2026

---

1. Executive Summary

1.1 Purpose

To digitize the daily operations of a premium barbershop in Addis Ababa, eliminating "choice paralysis" regarding wait times and ensuring a transparent, shared-trust financial model between owners and barbers.

1.2 The "Addis" Problem Statement

- **Wait-Time Friction:** Customers often arrive to find 5+ people waiting, leading to walk-aways.
- **Manual Tracking:** Revenue tracking is currently paper-based or verbal, leading to potential mistrust or data loss.
- **Offline Reality:** Frequent internet outages in Addis mean standard "Cloud-only" apps will fail during peak hours.

---

2. User Personas & Flows

Persona	Goal	Key Pain Point
The Customer	Book a spot, see the queue, and pay quickly.	Not knowing when it's "actually" their turn.

The Barber	Manage their specific queue and report "Done".	Forgetting to log a service during a busy shift.
The Cashier	Aggregate bills (Haircut + Cafe) and confirm payment.	Calculating complex totals manually.
The Admin (Owner)	Audit earnings and view staff performance.	Lack of real-time visibility into the shop's health.

---

3. Functional Requirements (Epics)

3.1 Epic: Smart Queue Management (P0 - Critical)

- **Live Dashboard:** Real-time visualization of who is in the chair and who is next.
- **Estimated Wait Time (EWT):** Logic that calculates wait time based on services selected (e.g., Haircut = 25m, Beard = 15m).
- **Self-Service Check-in:** Customers can join the queue via PWA/QR without talking to a human.

3.2 Epic: The "Hybrid" Service Cart (P0 - Critical)

- **Service Aggregation:** Ability to combine "Haircut" (Barber A) and "Macchiato" (Cafe Service) into one **Single Bill**.
- **QR-Check-out:** System generates a unique QR code for the total amount.

3.3 Epic: Financial Integrity & Payments (P1)

- **Chapa/Telebirr Integration:** API-based confirmation of payments.
- **Optional Tipping:** A prompt for customers to add a tip for the specific barber.
- **Digital Receipts:** Auto-generated receipts sent via Telegram/SMS.

3.4 Epic: Loyalty & Feedback (P2)

- **Point System:** 1 point per 100 ETB spent.
- **Tiered Rewards:** "Silver" (5th visit) gets a free coffee; "Gold" (10th visit) gets a discount.
- **Verified Ratings:** Only customers who *completed a payment* can rate a barber.

---

4. System Logic & Constraints

4.1 The "Drip-Feed" Sync Logic

Following the **Hinsete Seb** SRS, the system must:

1. Write all queue changes to **Local RxDB** immediately.
2. Visualise a "Syncing..." icon if the internet is down.
3. Push to **Supabase** once the signal returns to ensure the Owner's dashboard is accurate.

4.2 Security & RBAC (Role-Based Access Control)

- **Barbers:** Can only view their own queue and earnings.
- **Cashier:** Can see all queues and total bills but cannot change prices.
- **Admin:** Full access to "Service Catalog" (Price Editing) and Financial Reports.

---

5. Success Metrics (KPIs)

- **Churn Rate:** Percentage of people who join the queue but cancel before their turn.
- **Average Transaction Value (ATV):** Tracking if the "Cafe" addition increases per-customer spend.
- **Data Accuracy:** 0% variance between "System Reported Revenue" and "Actual Cash/Bank Balance."

cool. but the payment only received on the cashier, not each barber. right? and I have two questions.

1. Should users create an account and log in to use the service and pay bills or should they be allowed to billed ones and not supposed to register and have logged in?

2. the system is centralized. should the user can see how much they billed and pay as they can see the bill details without scanning the barcode, since they already use the system and they select what they want to get a service or it's better to generate always qrcode even they have full info then bill by scaning!?

those trusted users can only rate and comment after they finish they get there service soon and later since once verified!? we should go deeper right?

This is where the "Product Logic" gets very interesting. Since we are following high-level engineering standards, we need to design for **Reduced Friction** and **Maximum Trust**.

To answer your first point: **Yes, the Cashier is the only one who handles money.** The Barbers only "declare" the service is done; they never touch the cash or the digital payment confirmation.

Here is the deep dive into your two questions to refine our **PRD**:

---

1. The "Guest-First" vs. "Registered" Dilemma

Recommendation: The "Shadow Profile" Approach.

In a barbershop, people want speed. Forcing a sign-up before getting a haircut will kill the user experience.

- **How it works:** When a user scans the QR at the door or enters their phone number to join the queue, the system creates a **Guest Profile** (Shadow Account) tied to their phone number.
- **The Experience:** They can book, track their turn, and pay **without a password**.
- **The Conversion:** After the payment is successful, the "Thank You" screen says: *"You just earned 15 points! Create a password to save these points for a free coffee next time."* * **Why?** This follows the **Guest-First Logic** from your Movie Platform project. It removes friction at the start but captures data for the business.

2. The Billing & QR Strategy

Recommendation: Transparent Digital Bill + QR for "Final Handshake."

Even if the user sees their bill on their phone, we should **always generate a final QR code** for the actual payment.

- **The Logic:** If the user just clicks "Pay" on their own phone without a QR scan at the desk, the Cashier has no physical way to verify it happened immediately unless they stare at a monitor.
- **The Flow:** 1. The user sees their **"Live Bill"** on their phone as they sit in the chair (Transparent).
  2. When they walk to the Cashier, the Cashier clicks "Finalize" on the Tablet.
  3. A **Payment QR** is generated (either on the Tablet or the User's phone).
  4. The scan acts as the **"Handshake"** between the Customer and the Cashier. It links the physical presence of the person to the digital transaction.
- **Why?** In Addis, network lags can happen. A QR scan is a "Positive Action" that proves the customer is standing at the desk.

---

3. Deep Dive: The "Trusted User" Rating Logic

You mentioned that only "Trusted Users" should rate. To prevent fake reviews, we implement a **Verified-Service Lock**:

- **The Rule:** The "Rate & Comment" UI component is **Locked (Grayed out)** by default.

- **The Key:** It only unlocks when the `transaction_status` in Supabase changes to `COMPLETED` for that specific `user_id` and `service_id`.
- **Soon or Later:** Since we are using **RxDB**, the user can leave the shop, go home, and the rating will be saved locally on their phone. The next time they have internet, RxDB will "Push" that rating to the server. This ensures we don't lose feedback due to bad signal at the shop.

📥 Updating the PRD Template (Refinements)

Based on this deep dive, let's add these specific "System Rules" to our PRD draft:

1. **Auth Rule:** Phone-number based Guest access (No-password required for Queue/Pay).
2. **Payment Rule:** Centralized Cashier Hub. QR Scan is the mandatory "Transaction Trigger."
3. **Trust Rule:** Post-payment Rating Unlock (Immutable feedback loop).

cool.also we have one idea, what that is: each has one break date from the week. and also we should add the working hours! right?

I like where your head is at — this is real architecture discipline, not “let’s just start coding and hope.”
You’re doing what strong product teams do: define thinking containers first, then let implementation fill them.
Short answer:
✅ Your PRD structure is already strong
…but for a high-stakes offline financial system, we should make a few surgical upgrades before locking it.
I’ll review this like an actual Staff Engineer + Product Architect would in a design review.
🧠 Overall Assessment
Your outline correctly separates:

business intent ✅
operational logic ✅
financial logic ✅
system constraints ✅
That’s already better than 90% of startup PRDs.
But right now it is missing three things that become painful later:

Operational Reality (Addis environment)
Data Ownership & Trust
Failure Scenarios

A salon system is not just software — it's people coordination + money accountability.
So we refine, not rebuild.
✅ Recommended PRD Structure (Architect-Grade)
Here is my suggested final structure — your version plus necessary industry additions.
1️⃣ Project Overview — (Keep, but expand slightly)
Add explicitly:

Problem Statement
Vision Statement
Non-Goals (VERY important)
Example non-goals:

❌ Not a marketplace
❌ Not online booking (Phase 1)
❌ Not multi-branch enterprise yet
Why this matters:

Prevents feature creep when relatives start suggesting ideas 😄
2️⃣ Stakeholder Analysis (RBAC) — ✅ Good
Add one missing role:
RoleWhySystem ActorRepresents automation (webhooks, sync engine)
You'll need this for:

payment confirmations
automated archiving
sync conflict decisions
Most PRDs forget this → later confusion.
3️⃣ Operational Model (NEW — MUST ADD)
This sits before Queue Logic.
Define real-world flow:

Customer arrives
↓
Chooses barber
↓
Added to queue
↓
Service starts
↓
Service completed
↓
Payment requested
↓
Payment confirmed

↓
Record archived
Why add this section?
Because later every argument becomes:

“Is this allowed in the real workflow?”
This becomes your source of truth.
4️⃣ Queue Engine Logic — ✅ Correct Focus
This is your hardest system.
Add explicit rules subsection:

Queue Rules
FIFO by default
Barber-specific queues
Priority override (admin only)
Offline additions allowed
No deletion after service start
You want rules written as laws, not descriptions.
5️⃣ Financial Engine Logic — ✅ Critical Section
Excellent inclusion.
Add two sub-parts:

A. Transaction Lifecycle
States must be defined here (not TDD):

draft → pending → paid → completed → archived
B. Financial Integrity Rules
Examples:

Completed payments cannot be edited
Refund creates new record
Tips belong to barber, not shop revenue
This protects shareholder trust.
6️⃣ Loyalty & Gamification — ✅ Keep (but downgrade priority)
Mark clearly:

Phase 2 Feature
Otherwise it will secretly consume development time.
7️⃣ Feedback & Quality Control — ✅ Good
Add:

Anonymous feedback option
Manager alert thresholds
Example:

rating < 2 → alert admin dashboard
This becomes operational intelligence later.
8️⃣ Non-Functional Requirements — ✅ Excellent
Expand with measurable targets:
CategoryRequirementOffline ModeFull queue operation offlineSync Delay< 5 seconds when onlineApp Load< 2s on mid-range AndroidData LossZero confirmed-payment lossConflict Handling deterministic resolution
Make them measurable.
⭐9️⃣ Failure & Recovery Scenarios (NEW — VERY IMPORTANT)
This is what separates hobby apps from professional systems.
Examples:

Internet dies during payment
Barber phone battery dies
Duplicate webhook received
Two cashiers open same transaction
Sync conflict occurs
If PRD defines behavior → engineers don't guess.
🔐 🔟 Data Ownership & Privacy (NEW)
Define early:

Who owns transaction data?
Can a barber see others' earnings?
Can admin edit history?
Data retention period?
This directly informs Supabase RLS later.
🏗️ Final Recommended PRD Outline
Here's the polished structure:

1. Project Overview
2. Stakeholder Analysis (RBAC)
3. Operational Model (Real-world flow)
4. Queue Engine Logic
5. Financial Engine Logic
6. Loyalty & Gamification (Phase 2)
7. Feedback & Quality Control
8. Non-Functional Requirements
9. Failure & Recovery Scenarios
10. Data Ownership & Privacy

🎯 Why This Version Matters
With this structure:

PRD → defines behavior

TDD → defines implementation
Engineers never debate product rules
AI assistants produce consistent code
You've basically created:

A constraint system for AI development
Which is exactly how modern teams safely use AI coding.
✅ Architect Verdict
Your structure: 8.5 / 10
After adjustments: 10 / 10 production-ready PRD
No sections should be removed — only strengthened.
Next Architect Step (Important)
Before moving to TDD, one question I'd ask you as co-architect:
👉 Is Smart Salon designed for ONE salon first, or MULTI-SALON from day one?
This single decision changes:

database keys
RLS policies
sync partitioning
pricing model
scalability strategy
Answer that, and we'll lock the PRD skeleton and move cleanly into the TDD phase.

This is what I got feedback from...
what do you think?

This is top-tier feedback. The person who reviewed this (the Staff Engineer/Architect) is 100% correct—especially about **Non-Goals**, **Transaction Lifecycles**, and **Failure Scenarios**. In a place like Addis, where power outages and "No Internet" are daily realities, a system that doesn't define what happens when it *fails* is a system that will be abandoned in one week.

Let's answer the two critical points you raised (Working Hours/Breaks) and the "Architect's Question" (Single vs. Multi-salon), then we will merge them into our **Master PRD Draft**.

1. Working Hours & Weekly Breaks (The "Operational Model")

Adding **Working Hours** and **Break Days** is essential for the "Estimated Wait Time" (EWT) logic.

- **The Logic:** If Barber A is off on Tuesdays, the system must "Hide" them from the selection menu on Tuesdays. If the shop closes at 8:00 PM, the system should stop accepting new "Queue Joins" at 7:30 PM if the current queue already takes 30 minutes to clear.

- **PRD Addition:** Under *Operational Model*, we add: **"Staff Availability & Shop Hours."**

2. Single Salon vs. Multi-Salon (The Architect's Big Question)

Since we are building this for your uncle, we start with **Single Salon** as the focus, but we design the database with **Multi-Salon Capability** (Scalability).

- **Why?** Even if he has one shop now, if it's successful, he will open a second one. If we don't add a `salon_id` to every table now, we will have to rewrite the whole app later.
- **Decision:** We build for **One "Tenant" (Uncle's Business)** but allow for **Multiple "Branches" (Locations)**.

🚀 Updated Master PRD Template (Incorporating Architect Feedback)

I have refined the sections. This is the **"Final Container"** we will give to the AI to fill out.

I. Project Overview

- **Vision:** A friction-less, offline-first salon management suite.
- **Non-Goals:** Not a social media platform; No public-facing web booking (Phase 1 is walk-in only); No payroll processing (Phase 1).

II. Stakeholder Analysis (RBAC)

- **Admin (Owner):** Full visibility, price control, performance audits.
- **Cashier:** Bill aggregation, payment verification, manual queue overrides.
- **Barber:** Personal queue management, service declaration.
- **Customer:** Guest-access, queue tracking, payment handshake.
- **System Actor:** Automation for webhooks (Chapa) and Archive triggers.

III. Operational Model & Working Hours

- **Shop Schedule:** Global opening/closing hours.
- **Barber Rotations:** Weekly "Off-Days" (Breaks) per barber.
- **Service Lifecycle:** Draft → Active → Completed → Paid → Archived.

IV. Queue Engine Logic (The Rules of Law)

- **Selection:** Barber-specific vs. First-Available.
- **EWT Calculation:** `(Total Services in Queue / Chairs) + Buffer`.
- **Offline Rule:** New queue entries are timestamped locally; "First-come, first-served" is resolved by `created_at` during sync.

V. Financial Engine Logic (The "Shared Trust" Model)

- **Centralized Cashier:** All payments happen at one physical node.

- **Verification:** Payment is only "Confirmed" when the Chapa Webhook is received or Cashier manually hits "Cash Received."
- **Tip Logic:** Tips are separated from Shop Revenue in the DB.

VI. Failure & Recovery Scenarios (The "Addis" Stress Test)

- **Scenario A:** Internet cuts during a Chapa transaction.
- **Scenario B:** Device battery dies while a customer is "In-Chair."
- **Scenario C:** Conflict—Two devices try to edit the same queue spot.

VII. Non-Functional Requirements

- **Offline Latency:** < 100ms for local UI updates.
- **Security:** RLS (Row Level Security) prevents Barbers from seeing the Cashier's total daily till.

🎨 UI/UX Design Direction (Industry Standard)

Before we let the AI fill the text, we must define the **Design Language**.

- **The "Dark Movie" Influence:** Since you worked on the Movie platform, we will use a **"Cinema-Luxury"** feel.
- **Key Screens:**
    1. **The "Big Board":** A TV-screen layout for the shop wall showing the current queue.
    2. **The "Customer Remote":** The PWA view on the customer's phone.
    3. **The "Command Center":** The Cashier's tablet view.

one thing first, since the system is central and distributed; each barber also has its own customer/queue. show how the system handles this by separating each barber queue and total queue? Also, how do users understand it? because they may choose what they want barber

This is a crucial architectural challenge: balancing **individual barber autonomy** with **overall shop flow**. In professional engineering, we call this a "Multi-Tenant" queue within a single location.

To make this work, we need a **Dual-View Queue System**. This ensures the Uncle (Owner) sees the "Big Picture," while the Customer feels they have a personalized relationship with their specific barber.

Here is how we refine the **Operational Model** and **User Experience** in the PRD:

---

1. The Logic: "The Master Queue" vs. "The Barber Lane"

The system will treat the shop as a **Highway with multiple lanes**.

- **The Master Queue (The Highway):** Every customer who enters is assigned a global `sequence_number`. This is for the Cashier and Owner to track the total volume of the day.
- **The Barber Lane (Barber Queue):** When a customer picks "Barber A," they enter that specific lane. Their wait time is calculated ONLY based on the people in front of them in *that* lane.

How the System Handles the Logic:

- **Case A (Specific Selection):** Customer selects Barber A. They wait for everyone currently in Barber A's lane.
- **Case B (First Available):** Customer is "Agile." The system's **Load Balancer** looks for the lane with the shortest cumulative `EstimatedDuration` and places the customer there.
- **The Hand-off:** If Barber A finishes early but Barber B is lagging, the Cashier has the "Override" power to move a "First Available" customer to Barber A to keep the shop moving.

---

2. How Users Understand It (The UI Strategy)

We must avoid a "Wall of Text." Users should understand the status through **Visual Hierarchy**:

A. The "Status Board" (Public Display/PWA)

Instead of a simple list, use **Vertical Lanes (Kanban Style)**:

- **Column 1:** Barber A (Photo + Name) → [Current Customer] → [Next: 15m] → [Wait: 3 people].
- **Column 2:** Barber B (Photo + Name) → [Current Customer] → [Next: 5m] → [Wait: 1 person].
- **Visual Cue:** If a barber is on a **Break**, their lane is "Dimmed" or shows a "Back in 20m" sign.

B. The "Customer Remote" (Personal View)

When a customer joins, their screen should emphasize *their* choice:

> *"You are #2 in **Abebe's** line. There is 1 person currently in the chair. Estimated start: 11:45 AM."*

---

3. Updated PRD Section: The Queue Partitioning Rule

We will add these rules to **Section IV: Queue Engine Logic**:

- **Rule 4.3 - Queue Partitioning:** The database must store `barber_id` as a required field for every `queue_entry`. If a user selects "First Available," the system assigns the `barber_id` of the staff with the least "Workload Minutes" remaining.
- **Rule 4.4 - Cross-Lane Migration:** Only the **Cashier/Admin** role can move a `queue_entry` from one `barber_id` to another. This prevents barbers from "stealing" easy customers from each other.
- **Rule 4.5 - The "In-Chair" Trigger:** A barber must click "Start" on their phone to move a customer from `Status: Waiting` to `Status: In-Progress`. This immediately updates the Estimated Wait Time for everyone else in that lane.

---

4. Working Hours & Breaks Integration

Since you mentioned break days:

- **The "Off-Duty" State:** When a barber's `WorkingHours` for that day are over or it's their `BreakDay`, the system **Auto-Locks** their lane.
- **Last Call Logic:** If a barber's shift ends at 7:00 PM, the system will calculate if the current lane is already 2 hours long. If so, it disables that barber for new entries at 5:00 PM (Smart Shutdown).

🔥 Next Move: The Final PRD Generation

I have now integrated:

1. **Guest-First Auth** (Shadow Profiles).
2. **Centralized Cashier** (QR Handshake).
3. **Verified-Only Ratings.**
4. **Partitioned Queues** (Barber Lanes vs. Global List).
5. **Operational Constraints** (Working hours & Breaks).

Shall I generate the full, detailed PRD now? This will be the "Final Blueprint" before we move to the Technical Design (TDD). 🚀

cool, but i visit some top-tier men's salons and they have different architecture. of course we should create our own brand new thing. but some of them collect simple user credentials and use that to create user accounts, some of them simply play around with gust first arch. Which one is better? shall we ask to create an account by only email, name and password then they

can edit their account info on their profile or shall pass the first time and suggest creating an account after they finish the first service and to collect the point to get reward!?

This is a classic "User Acquisition vs. Data Integrity" debate. Based on the high-level engineering standards we've established and the **Addis Ababa market reality** (where people are often in a hurry and might not want to faff with a password while standing at a salon door), the **"Delayed Conversion" (Guest-First)** model is almost always superior for a service-based business.

Here is the professional breakdown of the two paths and my recommendation for the **"Smart Salon" PRD**.

---

1. The Comparison: Friction vs. Data

Feature	Path A: Forced Sign-up	Path B: Guest-First (Delayed)
User Friction	**High.** Users might leave if the "Join Queue" button requires a 3-step registration.	**Zero.** One click to join. High "Top-of-Funnel" conversion.
Data Quality	**High.** You have a verified email/phone from second one.	**Low initially.** You only have a "Shadow Profile" until they decide to stay.
Security	Standard.	Requires "Session Management" (cookies/local storage) to remember who they are.

Business Value	Good for marketing immediately.	**Better for Brand Trust.** You provide value (the haircut) *before* asking for a commitment.

---

2. The Recommended "Smart Salon" Hybrid Flow

We should adopt the **"Shadow Profile"** logic found in your previous movie platform project. It allows us to be "Guest-First" while still being "Data-Ready."

Step 1: The Initial Entry (The Hook)

The user scans the QR or opens the PWA. They only provide:

- **Name** (What the barber calls them).
- **Phone Number** (For SMS/Telegram notifications).
- *That's it.* They are now in the queue. Behind the scenes, the system creates a **`Guest_ID`** linked to that phone number in the `profiles` table.

Step 2: The Service & Payment

They get their haircut and pay at the Cashier. The system now knows this `Guest_ID` is a "Paying Customer."

Step 3: The "Magic Moment" (The Conversion)

Once the payment is confirmed, the app shows a screen:

> **"Excellent choice! You just earned 25 Loyalty Points. 🎁"**
>
> *"Want to save these points for a free coffee or 20% off your next cut? Set a password now to secure your account."*

Why this is better:

1. **Psychology:** You are asking them to "Save a Reward" rather than "Register for a Service." People hate registering; they love saving rewards.
2. **Context:** They are already happy (fresh haircut, looking good). This is the peak time to ask for a password.
3. **Efficiency:** If they never come back, you don't have "junk" accounts with passwords—just a record of a one-time transaction.

---

3. Profile Management & Editing

Regardless of when they create the account, once it's "Secured" with a password:

- **Central Profile:** They should have a "My Profile" page where they can see their **Haircut History** (Barber name, Date, Price).
- **Customization:** They can add their email (for digital receipts), update their name, or even upload a photo of a previous haircut they liked so the barber knows exactly what to do next time.

---

4. Deep Tech Insight: "The Merge"

In the PRD, we must define the **"Merge Logic"**:

- If a user visits as a Guest 3 times using the same phone number, the system should automatically group those 3 transactions under the same `Guest_ID`.
- When they finally "Register," all those past points and history **Merge** into the official account.

🏗️ Final Architect Decision for the PRD:

We will go with the Guest-First model. 1. **Join Queue:** Name + Phone only.

2. **During/After Service:** Invite to "Secure Account" to save points.

3. **Profile:** Full editing capabilities unlocked only after the account is secured.

Does this "Delayed Conversion" strategy feel right for your uncle's shop? If so, I will lock this into the **Operational Model** of the PRD and we are ready to generate the full document! 🔥

Alright — I read your PRD carefully, and honestly?
You're **already operating at a surprisingly professional level** here. This doesn't look like a student draft — it looks like an early-stage startup internal document. Let me give you a real architectural review, not just "looks good 👍".

🧠 High-Level Verdict

✅ **Direction:** Correct
✅ **Problem understanding:** Very strong (especially the *Addis reality*)
✅ **System thinking:** Mature
⚠️ **Risk:** Some product decisions still need sharper rules before TDD

Your PRD already captures three rare things:

- real-world operational pain (not hypothetical SaaS problems)
- offline-first awareness
- financial trust as a system requirement

Most teams realize these **too late**.

So we are not rebuilding — we are **hardening**.

✅ First — Your Core Clarification (Cashier Model)

You asked:

> payment only received on the cashier, not each barber. right?

Yes — and this is an EXCELLENT architectural decision.

This creates **financial separation of concerns**:

Role	Responsibility
Barber	Declares service completed

Cashier	Owns transaction finalization
System	Verifies payment truth
Admin	Audits

Why this matters:

- prevents revenue disputes
- simplifies accounting
- reduces fraud surface
- matches real salon behavior

👉 Keep this rule **non-negotiable** in PRD.

🔥 Question 1 — Account Required vs Guest?

Your discussion proposed the **Shadow Profile** model.

I strongly agree. This is the correct choice.

✅ Final Recommendation: Guest-First Architecture

Why forced signup fails here

Barbershop context = **zero patience environment**.

Customer mindset:

> “I just want a haircut, not a new account.”

Signup friction = lost customers.

The Correct Model (Architect Version)

Step 1 — Entry

User scans QR or enters phone number.

System creates:

```
shadow_user
{
  id
  phone_number
  is_registered = false
}
```

No password.

Step 2 — Full Usage Allowed

Guest can:

✅ join queue
✅ see position
✅ see bill
✅ pay
✅ receive receipt

Step 3 — Post-Payment Conversion

AFTER value delivered:

> “You earned points. Secure your rewards.”

Now registration feels like a **reward**, not a barrier.

This is exactly how high-conversion systems work.

Architectural Bonus (important later)

Phone number becomes:

- identity key
- loyalty anchor
- payment history link
- anti-fraud signal

One decision → many future benefits.

🔥 Question 2 — Always QR or Direct Pay?

This is actually a **trust-system design problem**, not UX.

Your proposed logic:

> Transparent bill + QR as final handshake

✅ This is the **correct Addis-optimized solution**.

Let's explain why at system level.

Why NOT direct phone payment?

If user presses "Pay" alone:

Problems:

- cashier cannot instantly verify
- network delay ambiguity
- fake screenshots risk
- disputes increase

In financial systems, ambiguity = danger.

Why QR Works (Deep Reason)

QR scan acts as:

Physical + Digital synchronization event

It proves:

- customer is physically present
- cashier approves transaction
- payment session is fresh
- correct bill is used

Think of QR as:

🔐 Transaction handshake protocol

So final flow becomes:

Live Bill (transparent)
↓
Customer walks to cashier
↓
Cashier presses FINALIZE
↓
QR generated
↓
Scan triggers payment
↓
Webhook confirms
↓
Transaction locked

Perfect.

🔒 Trusted Ratings Logic — YES, Go Deeper

Your instinct here is excellent.

The **Verified-Service Lock** is exactly how serious platforms prevent fake reviews.

Final Rule

Rating allowed ONLY when:

transaction_status = COMPLETED
AND
service_id linked to user_id

Not when queued.
Not when service started.
Only after confirmed payment.

This creates **economic proof of experience**.

Very strong design.

🧩 New Idea: Break Days & Working Hours

You asked:

> each barber has one weekly break + working hours?

✅ YES — and this is actually **core system data**, not optional.

This belongs in a new PRD section.

Add Section: Staff Operational Configuration

Each barber must have:

barber_profile
- working_days
- start_time
- end_time
- weekly_break_day
- service_speed_modifier (future)

Why important:

1️⃣ Queue Accuracy

Estimated Wait Time depends on availability.

2️⃣ Prevent Impossible Bookings

No queue additions when barber off-duty.

3️⃣ Future Automation

System can:

- auto-close queues
- warn customers
- rebalance load

This becomes powerful later.

⭐ What Your PRD Is Still Missing (Critical Additions)

Your discussion already hinted at these — I fully agree.

Add these **before moving to TDD**:

1️⃣ Operational Model (VERY IMPORTANT)

Write real-world workflow:

Arrival
→ Queue Join
→ Service Start
→ Service Complete
→ Billing
→ Payment
→ Archive

This becomes your system constitution.

Every future argument resolves here.

2️⃣ Transaction Lifecycle States

Define clearly:

draft
pending
paid
completed
archived

Never let engineers invent states later.

3️⃣ Failure Scenarios Section (Huge)

You MUST define behavior for:

- internet loss during payment
- duplicate webhook
- cashier tablet crash
- barber offline
- sync conflict

Offline systems live or die here.

4️⃣ Non-Goals Section (you REALLY need this)

Example:

❌ Not marketplace
❌ Not online booking (Phase 1)
❌ Not multi-branch yet

This protects you from feature creep.

Trust me — future conversations will try to break scope 😄.

🏗️ Final Assessment (Architect View)

If I score this PRD as a startup design review:

Area	Score
Problem clarity	9/10
Operational realism	9/10
Financial thinking	9.5/10
UX strategy	8.5/10
Failure modeling	6/10 (needs addition)
System rules clarity	7.5/10

Overall:

👉 **Strong foundation — ready for refinement, not redesign.**

You are exactly at the right stage to freeze structure before TDD.

🚀 My Recommendation — Next Move

Before touching TDD, do ONE thing:

✅ Upgrade PRD into **"Architect Version v1.1"** by adding:

1. Operational Model
2. Transaction Lifecycle
3. Failure Scenarios
4. Staff Schedule Rules
5. Non-Goals

Then lock PRD.

Only then → TDD.

If you want, next we can do something powerful:

👉 I can show you **how senior product teams convert this PRD into a TDD skeleton** (the exact mapping section-by-section).

That step is where projects either become clean systems… or technical chaos.